

تمرین سری سوم الگوریتم‌های معاملاتی

کوروش تقی‌پور پاسدار

۶ آذر ۱۴۰۳

فهرست مطالب

۲	بخش اول	۱
۲	Import Libraries and Data	۱.۱
۳	Bollinger Band	۲.۱
۳	فرمول ریاضی	۱.۲.۱
۳	پیاده‌سازی	۲.۲.۱
۴	تست پیاده‌سازی	۳.۲.۱
۵	Keltner Channel	۳.۱
۵	فرمول ریاضی	۱.۳.۱
۵	پیاده‌سازی	۲.۳.۱
۶	تست پیاده‌سازی	۳.۳.۱
۷	RSI	۴.۱
۷	فرمول ریاضی	۱.۴.۱
۷	پیاده‌سازی	۲.۴.۱
۸	تست پیاده‌سازی	۳.۴.۱
۸	Backtest	۵.۱
۱۰	قسمت بررسی شرط	۱.۵.۱
۱۰	بررسی و بستن پوزیشن sell در صورت وجود	۲.۵.۱
۱۰	بازکردن پوزیشن buy	۳.۵.۱
۱۱	قسمت بررسی شرط	۴.۵.۱
۱۱	بستن پوزیشن‌های باز Buy	۵.۵.۱
۱۱	باز کردن پوزیشن Sell	۶.۵.۱
۱۱	نتایج Backtest	۶.۱
۱۴	معیارهای ارزیابی مختلف	۷.۱
۱۶	تغییر در استراتژی	۸.۱
۱۶	تغییر در مقدار Oversold و Overbought	۱.۸.۱
۱۶	بخش دوم	۲
۱۶	Backtest تابع	۱.۲
۱۷	بخش ابتدایی تابع	۱.۱.۲
۱۷	باز کردن پوزیشن Sell	۲.۱.۲

۱۸		Buy کردن پوزیشن	۳.۱.۲
۱۸		بررسی سیگنال‌های خرید یا فروش	۴.۱.۲
۱۹		Backtest اجرای تابع	۵.۱.۲

۱ بخش اول

۱.۱ Import Libraries and Data

در ابتدا مطابق با تصویر زیر، کتابخانه‌ها و داده‌های مربوط به دو رمزارز Bitcoin و Ethereum در دو interval یک روزه و پنج روزه در بازه زمانی اول فوریه ۲۰۲۳ تا اول فوریه ۲۰۲۴ لود می‌شوند.

```

Import Library and Data

1 import yfinance as yf
2 import pandas as pd
3 import numpy as np
4 from datetime import datetime, timedelta
5 import matplotlib.pyplot as plt
482] ✓ 0.0s Python

1 BTC_Ticker = "BTC-USD"
2 ETH_Ticker = "ETH-USD"
483] ✓ 0.0s Python

1 start_date = datetime(2023, 2, 1)
2 end_date = datetime(2024, 2, 1)
484] ✓ 0.0s Python

1 BTC_1d = yf.Ticker(BTC_Ticker).history(interval="1d", start=start_date, end=end_date)
2 ETH_1d = yf.Ticker(ETH_Ticker).history(interval="1d", start=start_date, end=end_date)
485] ✓ 0.0s Python

1 BTC_5d = yf.Ticker(BTC_Ticker).history(interval="5d", start=start_date, end=end_date)
2 ETH_5d = yf.Ticker(ETH_Ticker).history(interval="5d", start=start_date, end=end_date)
486] ✓ 0.0s Python

```

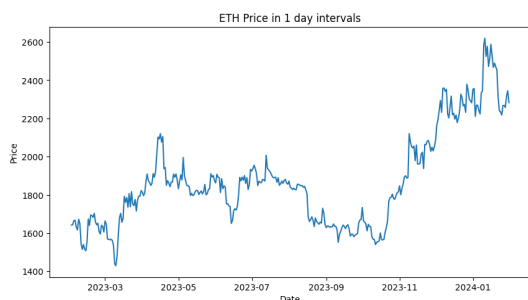
سپس در کد زیر، نمودار قیمت‌های این دو رمزارز نمایش داده می‌شوند.

```

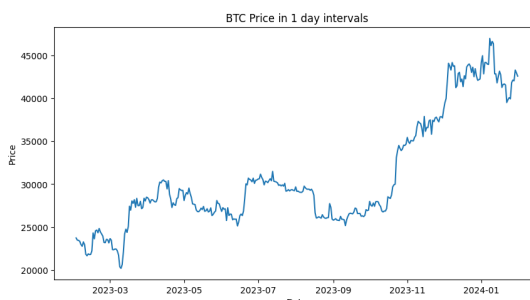
1 plt.figure(figsize=(10,24))
2
3 plt.subplot(4,1,1)
4 plt.plot(BTC_1d.Close)
5 plt.xlabel("Date")
6 plt.ylabel("Price")
7 plt.title("BTC Price in 1 day intervals")
8
9 plt.subplot(4,1,2)
10 plt.plot(ETH_1d.Close)
11 plt.xlabel("Date")
12 plt.ylabel("Price")
13 plt.title("ETH Price in 1 day intervals")
14
15 plt.subplot(4,1,3)
16 plt.plot(BTC_5d.Close)
17 plt.xlabel("Date")
18 plt.ylabel("Price")
19 plt.title("BTC Price in 5 day intervals")
20
21 plt.subplot(4,1,4)
22 plt.plot(ETH_5d.Close)
23 plt.xlabel("Date")
24 plt.ylabel("Price")
25 plt.title("ETH Price in 5 day intervals")
26
27 plt.show()
✓ 0.5s

```

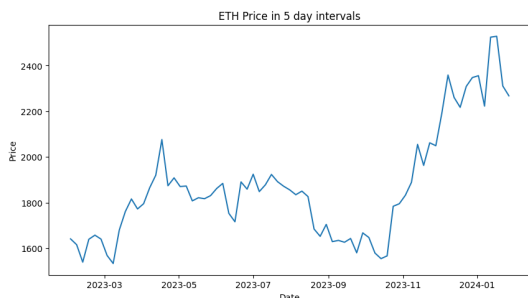
در تصاویر زیر نیز نمودار قیمت‌های رمزارزهای BTC و ETH در دو بازه زمانی ۱ روزه و ۵ روزه نمایش داده می‌شود.



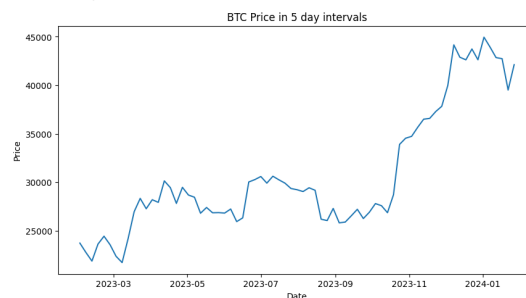
(ب) نمودار قیمت رمزارز ETH در بازه‌های زمانی ۱ روزه



(آ) نمودار قیمت رمزارز BTC در بازه‌های زمانی ۱ روزه



(د) نمودار قیمت رمزارز ETH در بازه‌های زمانی ۵ روزه



(ج) نمودار قیمت رمزارز BTC در بازه‌های زمانی ۵ روزه

۲.۱ Bollinger Band

۱.۲.۱ فرمول ریاضی

در ادامه اندیکاتور Bollinger Band تعریف می‌شود. در این اندیکاتور، ما باندهای بالایی، میانی و پایینی را تعریف و محاسبه می‌کنیم.
باند بالایی از فرمول

$$SMA + Std \times Correlation$$

و باند پایینی از فرمول

$$SMA - Std \times Correlation$$

محاسبه می‌شود. همچنین باند میانی با SMA برابر است.

۲.۲.۱ پیاده‌سازی

حال برای محاسبه اندیکاتور Bollinger Band، ما ۴ تابع زیر را تعریف می‌کنیم:

۱. **SMA** در این تابع، ما SMA چند شمع قبلی را حساب می‌کنیم. تعداد این شمع‌ها توسط متغیر period تعیین می‌شود.

۲. **STD** در این تابع، ما انحراف معیار را محاسبه می‌کنیم.

۳. **UpperBound** در این تابع، ما با استفاده از توابع SMA و STD باند بالایی اندیکاتور را حساب می‌کنیم.

۴. **LowerBound** در این تابع، ما با استفاده از توابع SMA و STD باند پایینی را حساب می‌کنیم.

در تصویر زیر، پیاده‌سازی اندیکاتور Bollinger Bond نشان داده شده است.

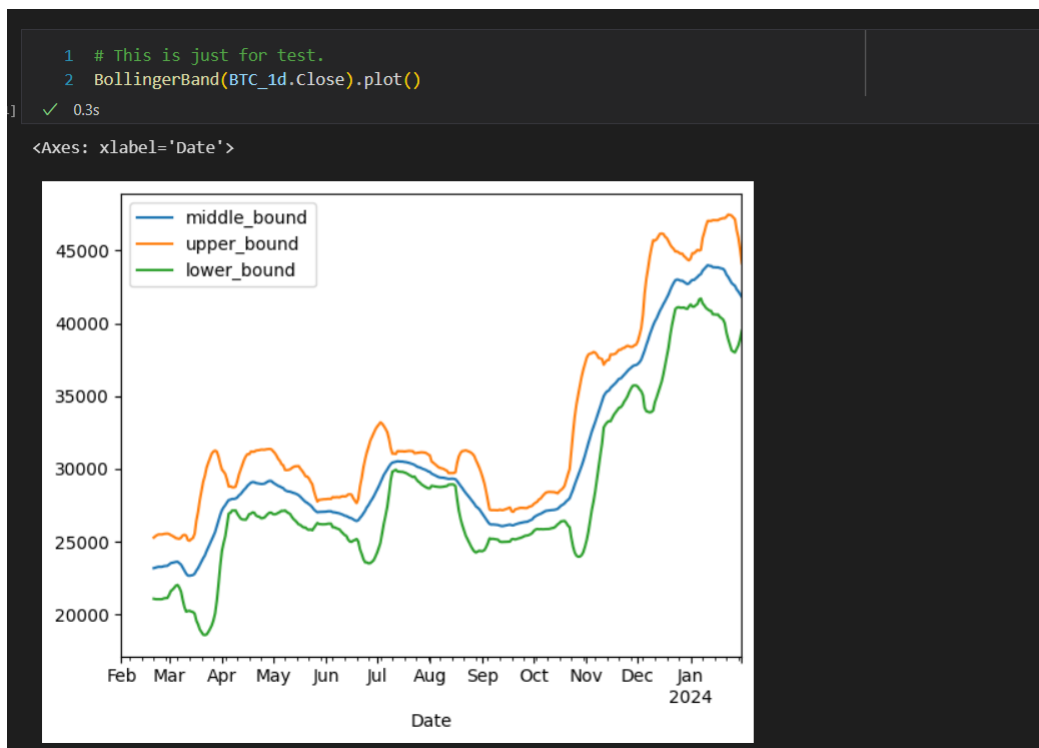
```
1 def BollingerBand(stock):
2     def SMA(series):
3         return series.rolling(period).apply(mean)
4
5     def STD(series):
6         return series.rolling(period).apply(stdev)
7
8     def UpperBound(series):
9         std = STD(series)
10        sma = SMA(series)
11        return sma + std * correlation
12
13    def LowerBound(series):
14        std = STD(series)
15        sma = SMA(series)
16        return sma - std * correlation
17
18    middle_bound = pd.Series(SMA(stock), name="middle_bound")
19    upper_bound = pd.Series(UpperBound(stock), name="upper_bound")
20    lower_bound = pd.Series(LowerBound(stock), name="lower_bound")
21    bollinger_band = pd.concat([middle_bound, upper_bound, lower_bound], axis="columns")
22
23    return bollinger_band
```

✓ 0.0s

شکل ۱: اندیکاتور Bollinger Bond

۳.۲.۱ تست پیاده‌سازی

برای نمونه، در تصویر زیر اندیکاتور Bollinger Bond برای قیمت Close نمودار رمزارز BTC در بازه‌های زمانی ۱ روزه نشان داده شده است.



شکل ۲: اندیکاتور Bollinger Band برای رمزارز BTC در بازه‌های زمانی ۱ روزه

۳.۱ Keltner Channel

۱.۳.۱ فرمول ریاضی

حال اندیکاتور Keltner Channel تعریف می‌شود. در این اندیکاتور، ما کانال‌های بالایی و پایینی را بصورت زیر تعریف می‌کنیم.

$$UpperBound = SMA + ATR \times Correlation$$

$$LowerBound = SMA - ATR \times Correlation$$

همچنین مقدار ATR بصورت زیر بدست می‌آید.

$$ATR_{init} = \frac{\sum_{i=1}^{14} TR_i}{14}$$

$$ATR_{j+1} = \frac{TR_j + (n - 1) \times ATR_j}{n}$$

که در اینجا منظور از n، تعداد روزهای دوره است.

۲.۳.۱ پیاده‌سازی

حال برای محاسبه باندهای بالایی و پایینی اندیکاتور Keltner Channel، ما از توابع زیر استفاده می‌کنیم:

۱. SMA کارکردی مشابه تابع SMA در اندیکاتور Bollinger Bound دارد.

۲. return_max از میان مقادیر $\text{High} - \text{Low}$ ، $\text{High} - \text{PrevClose}$ و $\text{Low} - \text{PrevClose}$ بیشترین مقدار را انتخاب می‌کند.

۳. TR به محاسبه مقادیر $\text{High} - \text{Low}$ ، $\text{High} - \text{PrevClose}$ و $\text{Low} - \text{PrevClose}$ می‌پردازد.

۴. ATR با استفاده از تابع TR به محاسبه ATR می‌پردازد.

۵. UpperBound با استفاده از مقادیر sma و atr به محاسبه باند بالایی می‌پردازد.

۶. LowerBound با استفاده از مقادیر sma و atr به محاسبه باند پایینی می‌پردازد.

حال در تصویر زیر، پیاده‌سازی این اندیکاتور نشان داده شده است.

```

13 high_low_diff = series.High - series.Low
14 high_prevclose_diff = series.High - series.Close.shift(periods=1)
15 low_prev_close_diff = series.Low - series.Close.shift(periods=1)
16 TR = pd.concat(
17     [high_low_diff, high_prevclose_diff, low_prev_close_diff],
18     axis=1,
19 )
20 TR.columns = ["high_low_diff", "high_prevclose_diff", "low_prev_close_diff"]
21 TR["TR"] = TR.apply(return_max, axis=1)
22 return TR["TR"]
23
24 def ATR(series, n):
25     tr = TR(series)
26     prev_atr = mean(tr[:14])
27     atr = pd.Series(np.nan, index=series.index)
28     for day in series.index[14:]:
29         current_atr = (tr.loc[day] + (n - 1) * prev_atr) / n
30         atr.loc[day] = current_atr
31         prev_atr = current_atr
32     return atr
33
34 def UpperBound(sma, atr):
35     return sma + atr * correlation
36
37 def LowerBound(sma, atr):
38     return sma - atr * correlation
39
40 atr = ATR(stock, n)
41 sma = SMA(stock.Close)
42 upper_bound = pd.Series(UpperBound(sma, atr), name="upper_bound")
43 lower_bound = pd.Series(LowerBound(sma, atr), name="lower_bound")
44 keltner_channel = pd.concat([upper_bound, lower_bound], axis="columns")
45 return keltner_channel

```

(ب) ادامه تابع اندیکاتور Keltner Channel

```

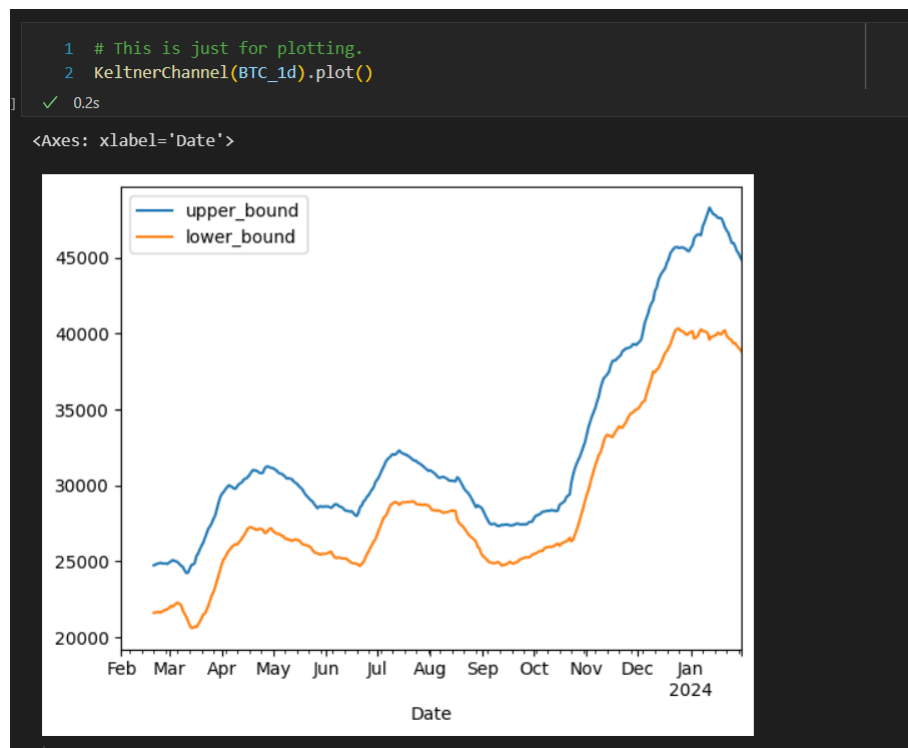
1 def KeltnerChannel(stock, n=14):
2     def SMA(series):
3         return series.rolling(period).apply(mean)
4
5     def return_max(series):
6         return max(
7             series["high_low_diff"],
8             series["high_prevclose_diff"],
9             series["low_prev_close_diff"],
10        )
11
12     def TR(series):
13         high_low_diff = series.High - series.Low
14         high_prevclose_diff = series.High - series.Close.shift(periods=1)
15         low_prev_close_diff = series.Low - series.Close.shift(periods=1)
16         TR = pd.concat(
17             [high_low_diff, high_prevclose_diff, low_prev_close_diff],
18             axis=1,
19         )
20         TR.columns = ["high_low_diff", "high_prevclose_diff", "low_prev_close_diff"]
21         TR["TR"] = TR.apply(return_max, axis=1)
22         return TR["TR"]
23
24     def ATR(series, n):
25         tr = TR(series)
26         prev_atr = mean(tr[:14])
27         atr = pd.Series(np.nan, index=series.index)
28         for day in series.index[14:]:
29             current_atr = (tr.loc[day] + (n - 1) * prev_atr) / n
30             atr.loc[day] = current_atr
31             prev_atr = current_atr
32         return atr

```

(آ) قسمت اول تابع اندیکاتور Keltner Channel

۳.۳.۱ تست پیاده‌سازی

در تصویر زیر، اندیکاتور Keltner Channel برای رمزارز BTC با بازه‌های زمانی ۱ روزه محاسبه و ترسیم شده است.



شکل ۳: اندیکاتور Keltner Channel برای رمزارز BTC در بازه‌های زمانی ۱ روزه

۴.۱ RSI

سپس به پیاده‌سازی RSI می‌پردازیم.

۱.۴.۱ فرمول ریاضی

$$RS = \frac{\text{average gain}}{\text{average loss}}$$

$$RSI = 100 - \frac{100}{1 + RS}$$

۲.۴.۱ پیاده‌سازی

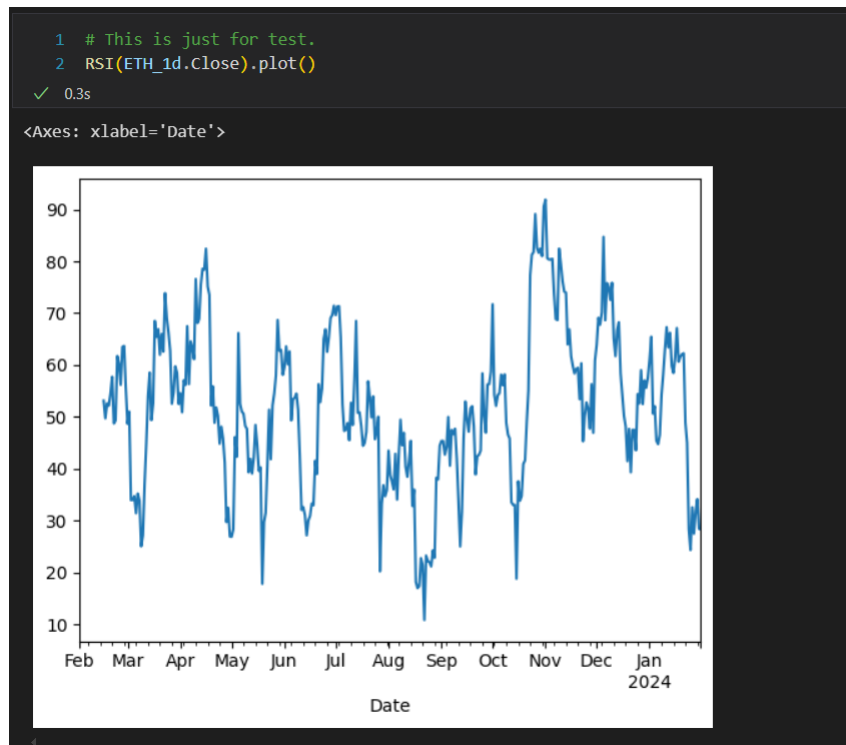
```

1 def RSI(series, n=14):
2     dUp, dDown = series.copy().diff(), series.copy().diff()
3     dUp[dUp < 0] = 0
4     dDown[dDown > 0] = 0
5     RoUp = dUp.rolling(n).mean()
6     RoDown = dDown.rolling(n).mean().abs()
7     RS = RoUp / RoDown
8     return 100 - (100 / (1 + RS))

```

✓ 0.0s

۳.۴.۱ تست پیاده‌سازی



شکل ۴: RSI محاسبه شده برای رمزارز ETH با بازه‌های زمانی ۱ روزه

۵.۱ Backtest

حال به تشریح تابع backtest می‌پردازیم.

در این تابع، ما مقادیر stock، init_fund و fund_rate را ورودی می‌گیریم. stock همان داده‌های قیمت رمزارز ما می‌باشد. init_fund هم سرمایه اولیه ما می‌باشد که با توجه به اشاره سوال به مقدار ۱۰۰۰، به طور پیشفرض برابر با ۱۰۰۰ می‌باشد. fund_rate نیز یک عدد در بازه صفر تا یک می‌باشد و اشاره دارد که در هر پوزیشن خرید یا فروش، چند درصد سرمایه را وارد کنیم.


```

1 def backtest(stock, init_fund=1000, fund_rate=1):
2     bollingerband_analysis = BollingerBand(stock.Close)
3     keltnerchannel_analysis = KeltnerChannel(stock)
4     rsi_analysis = RSI(stock.Close)
5
6     oversold = 30
7     overbought = 70
8
9     current_share = 0
10    current_fund = init_fund
11    buy_holding = False
12    sell_holding = False
13    sell_price = None
14
15    events_list = []
16    fund_per_day = pd.Series(0, index=stock.index)

```

شکل ۵: بخش اول تابع backtest

در ابتدای تابع، تحلیل‌های اندیکاتورهای Bollinger Band، Keltner Channel و RSI را بر روی قیمت رمزارز ورودی انجام می‌دهیم. سپس متغیرهای زیر را تعریف می‌کنیم:

- **oversold** مقدار oversold برای اندیکاتور RSI
- **overbought** مقدار overbought برای اندیکاتور RSI
- **current_share** تعداد سهام‌های خریداری شده
- **current_fund** سرمایه (نقدی) موجود
- **buy_holding** آیا پوزیشن buy باز داریم یا خیر
- **sell_holding** آیا پوزیشن sell باز داریم یا خیر
- **sell_price** در صورت وجود پوزیشن sell، قیمت سهام‌ها در زمان باز کردن پوزیشن چقدر بوده است.
- **events_list** لیستی برای ذخیره تاریخچه خرید و فروش پوزیشن‌ها

در ابتدای حلقه، ما به قیمت رمزارز در آن روز و همچنین تحلیل اندیکاتورهای Bollinger Band، Keltner Channel و RSI در آن روز را استخراج می‌کنیم. دقت شود در ابتدای تابع، ما تحلیل این اندیکاتورها را برای تمام زمان‌ها استخراج کردیم و در این قسمت ما به استخراج این اندیکاتورها برای امروز می‌پردازیم.

```

17 for day in stock.index:
18     stock_price = stock.loc[day].Close
19     b_price = bollingerband_analysis.loc[day]
20     k_price = keltnerchannel_analysis.loc[day]
21     rsi_rate = rsi_analysis.loc[day]

```

حال در این بخش ما به باز کردن پوزیشن buy می‌پردازیم.

```

22     # Buy
23     if (
24         not buy_holding
25         and stock_price <= b_price["lower_bound"]
26         and stock_price <= k_price["lower_bound"]
27         and rsi_rate <= oversold
28     ):
29         if sell_holding:
30             current_fund += current_share * sell_price
31             current_fund += (sell_price - stock_price) * current_share
32             current_share = 0
33             sell_holding = False
34             events_list.append(("b", day))
35             current_share += fund_rate * current_fund / stock_price
36             current_fund -= current_share * stock_price
37             buy_holding = True
38

```

شکل ۶: بخش بازکردن پوزیشن Buy

۱.۵.۱ قسمت بررسی شرط

در قسمت شرط، ما شروط گفته شده در صورت سوال را بررسی می‌کنیم. همچنین بررسی می‌کنیم آیا پوزیشن buy باز داریم تا از تکرار باز کردن پوزیشن جلوگیری کنیم.

۲.۵.۱ بررسی و بستن پوزیشن sell در صورت وجود

داخل شرط بررسی می‌کنیم آیا پوزیشن sell باز داریم یا خیر. در صورت باز بودن پوزیشن sell، پوزیشن sell را می‌بندیم. به اینصورت که در ابتدا سرمایه‌ای را که برای بازکردن پوزیشن صرف کرده بودیم را برمی‌گردانیم. سپس اختلاف قیمت در زمان باز کردن پوزیشن sell و بستن را حساب می‌کنیم تا در صورت افت قیمت، از روند نزولی بازار سود کسب کرده باشیم و در صورت صعود قیمت، از روند صعودی بازار با پوزیشن sell ضرر کنیم. سپس تعداد سهم‌های موجود را صفر کرده و مقدار sell_holding را برابر با False قرار می‌دهیم به این معنی که پوزیشن sell باز نداریم.

۳.۵.۱ بازکردن پوزیشن buy

در ادامه، ما روز و نوع پوزیشن باز شده را در لیست events_list ذخیره می‌کنیم. سپس تعداد سهم‌های قابل خریداری را با توجه به مقدار سرمایه نقدی موجود و درصد سرمایه قابل ورود (fund_rate) را محاسبه می‌کنیم. سپس هزینه آن را از سرمایه نقدی موجود کم می‌کنیم. همچنین متغیر buy_holding را برابر با True قرار می‌دهیم تا بدانیم هم‌اکنون پوزیشن buy باز داریم.

```

39     # Sell
40     elif (
41         not sell_holding
42         and stock_price >= b_price.loc["upper_bound"]
43         and stock_price >= k_price["upper_bound"]
44         and rsi_rate >= overbought
45     ):
46         if buy_holding:
47             current_fund += stock_price * current_share
48             current_share = 0
49             buy_holding = False
50         events_list.append(("s", day))
51         current_share += fund_rate * current_fund / stock_price
52         current_fund -= current_share * stock_price
53         sell_price = stock_price
54         sell_holding = True

```

شکل ۷: باز کردن پوزیشن Sell

حال به بخش بازکردن پوزیشن sell می‌پردازیم.

۴.۵.۱ قسمت بررسی شرط

در قسمت شرط، ما در ابتدا بررسی می‌کنیم آیا پوزیشن sell باز داریم یا خیر، تا بدینصورت از باز کردن مجدد پوزیشن Sell خودداری کنیم. سپس بررسی می‌کنیم آیا قیمت سهم امروز از باند بالایی اندیکاتور Bollinger Band و Keltner Channel و همچنین مقدار RSI از مقدار overbought بیشتر است یا خیر. در صورت صحیح بودن همه، وارد بدنه شرط می‌شویم.

۵.۵.۱ بستن پوزیشن‌های باز Buy

در ابتدا بررسی می‌کنیم آیا پوزیشن باز Buy داریم یا خیر. در صورت وجود، با ضرب تعداد سهم‌ها در قیمت امروز سهم و صفر کردن تعداد سهم‌ها، پوزیشن‌های باز Buy را می‌بندیم. همچنین متغیر buy_holding را برابر با False قرار می‌دهیم به این معنی که دیگر پوزیشن باز Buy نداریم.

۶.۵.۱ باز کردن پوزیشن Sell

در ابتدا روز و نوع پوزیشن باز شده را در events_list ذخیره می‌کنیم. سپس با توجه به سرمایه نقدی موجود، درصد ورود سرمایه و قیمت هر سهم، تعداد سهم‌های خریداری شده را محاسبه می‌کنیم. سپس قیمت آنها را از سرمایه‌ی نقدی کم می‌کنیم. همچنین متغیر sell_holding را برابر با True قرار می‌دهیم تا بدانیم هم‌اکنون پوزیشن باز sell داریم.

```

54     fund_per_day.loc[day] = current_fund + current_share * stock_price
55
56
57     return fund_per_day, events_list

```

در انتهای هر حلقه، مقدار سرمایه آن روز (نقدی و سهم‌ها) را محاسبه کرده و ذخیره می‌کنیم. همچنین در انتهای تابع، مقدار سرمایه در هرروز و همچنین تاریخ‌های خرید و فروش‌ها را برمی‌گردانیم.

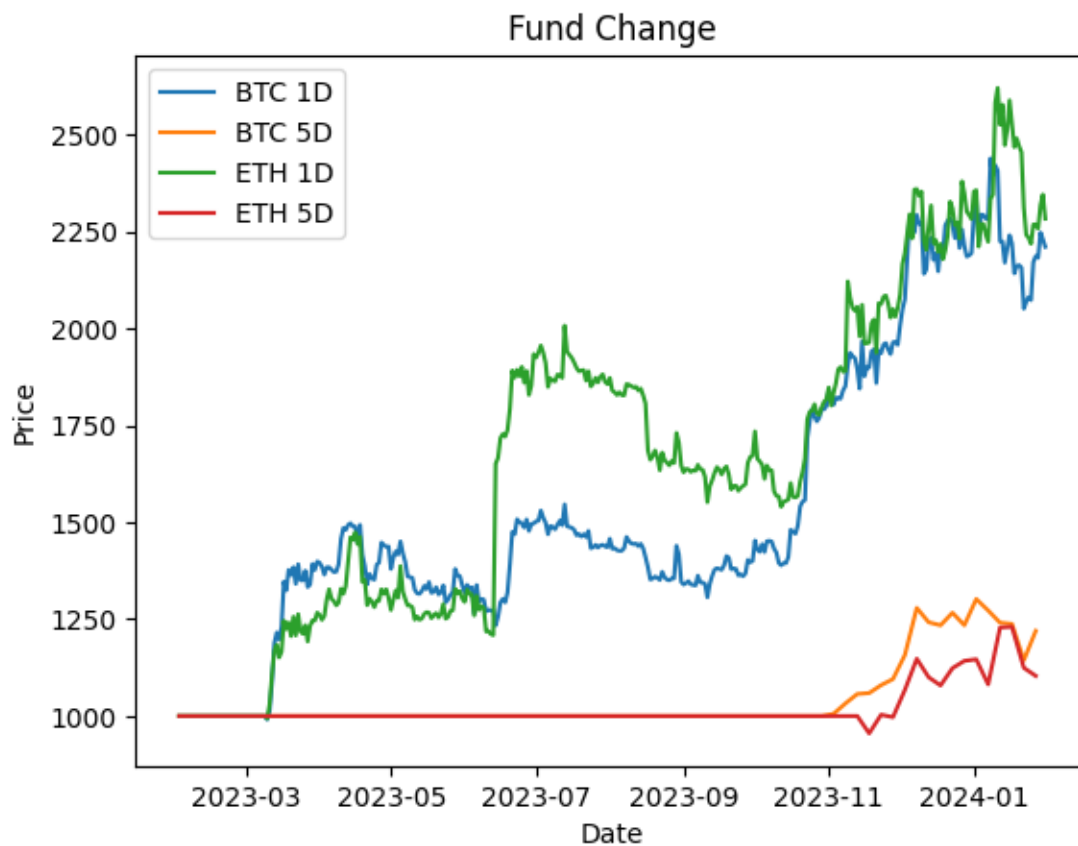
۶.۱ نتایج Backtest

در کد زیر، تابع backtest را برای هر یک از رمزارزهای BTC و ETH در دو بازه زمانی ۱ روزه و ۵ روزه نشان می‌دهد. همچنین $fundrate = 1$ نشان می‌دهد ما با تمام سرمایه وارد می‌شویم.

```
1 fund_rate = 1
✓ 0.0s

1 BTC_1D_Result, BTC_1D_Events = backtest(BTC_1d, fund_rate=fund_rate)
2 BTC_5D_Result, BTC_5D_Events = backtest(BTC_5d, fund_rate=fund_rate)
3 ETH_1D_Result, ETH_1D_Events = backtest(ETH_1d, fund_rate=fund_rate)
4 ETH_5D_Result, ETH_5D_Events = backtest(ETH_5d, fund_rate=fund_rate)
5
6 plt.plot(BTC_1D_Result, label="BTC 1D")
7 plt.plot(BTC_5D_Result, label="BTC 5D")
8 plt.plot(ETH_1D_Result, label="ETH 1D")
9 plt.plot(ETH_5D_Result, label="ETH 5D")
10 plt.title("Fund Change")
11 plt.xlabel("Date")
12 plt.ylabel("Price")
13 plt.legend()
14 plt.show()
✓ 4.0s
```

نتیجه در تصویر زیر ترسیم شده است.



همچنین در کدهای زیر، زمان‌های خرید و فروش در هر رمز ارزی نشان داده شده است.

```

4 plt.subplot(4,1,1)
5 plt.plot(BTC_1d.Close)
6 plt.xlabel("Date")
7 plt.ylabel("Price")
8 plt.legend()
9 for idx, event in enumerate(BTC_1D_Events):
10     color = 'red' if event[0] == 's' else 'blue'
11     plt.axvline(event[1], color=color, linestyle='--', alpha=0.4)
12 plt.title("BTC 1 day interval results")

```

(آ) رسم زمان‌های خرید و فروش برای رمزارز BTC در بازه‌های زمانی ۱ روزه

```

14 plt.subplot(4,1,2)
15 plt.plot(BTC_5d.Close)
16 plt.xlabel("Date")
17 plt.ylabel("Price")
18 plt.legend()
19 for idx, event in enumerate(BTC_5D_Events):
20     color = 'red' if event[0] == 's' else 'blue'
21     plt.axvline(event[1], color=color, linestyle='--', alpha=0.4)
22 plt.title("BTC 5 day interval results")

```

(ب) رسم زمان‌های خرید و فروش برای رمزارز BTC در بازه‌های زمانی ۵ روزه

```

24 plt.subplot(4,1,3)
25 plt.plot(ETH_1d.Close)
26 plt.xlabel("Date")
27 plt.ylabel("Price")
28 plt.legend()
29 for idx, event in enumerate(ETH_1D_Events):
30     color = 'red' if event[0] == 's' else 'blue'
31     plt.axvline(event[1], color=color, linestyle='--', alpha=0.4)
32 plt.title("ETH 1 day interval results")

```

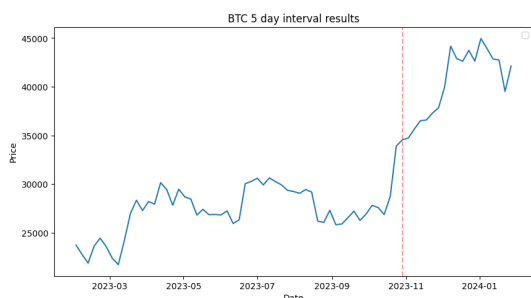
(ج) رسم زمان‌های خرید و فروش برای رمزارز ETH در بازه‌های زمانی ۱ روزه

```

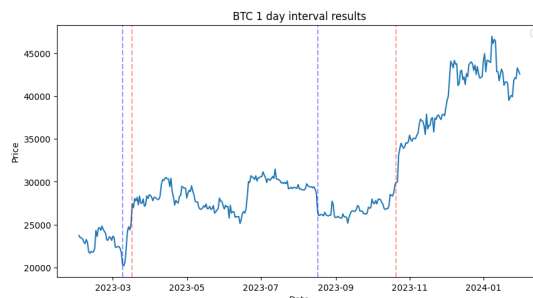
34 plt.subplot(4,1,4)
35 plt.plot(ETH_5d.Close)
36 plt.xlabel("Date")
37 plt.ylabel("Price")
38 plt.legend()
39 for idx, event in enumerate(ETH_5D_Events):
40     color = 'red' if event[0] == 's' else 'blue'
41     plt.axvline(event[1], color=color, linestyle='--', alpha=0.4)
42 plt.title("ETH 5 day interval results")

```

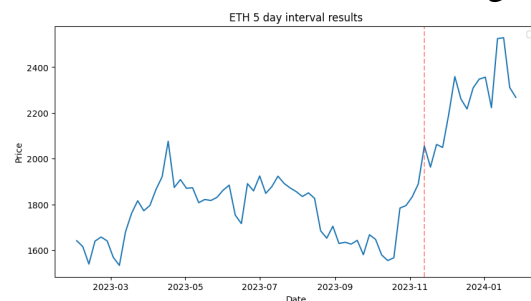
(د) رسم زمان‌های خرید و فروش برای رمزارز ETH در بازه‌های زمانی ۵ روزه



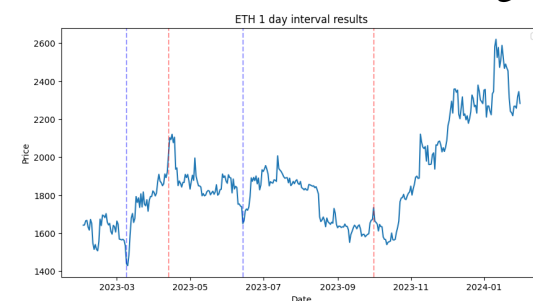
(و) زمان‌های خرید و فروش برای رمزارز BTC در بازه‌های زمانی ۵ روزه



(ه) زمان‌های خرید و فروش برای رمزارز BTC در بازه‌های زمانی ۱ روزه



(ح) زمان‌های خرید و فروش برای رمزارز ETH در بازه‌های زمانی ۵ روزه



(ز) زمان‌های خرید و فروش برای رمزارز ETH در بازه‌های زمانی ۱ روزه

قابل به ذکر است که ممکن است پوزیشنی در یک نمودار باز شود و تا انتهای نمودار به دلیل عدم فراهم شدن شرایط لازم، بسته نشود. به همین دلیل، بستن پوزیشن‌ها در انتهای نمودار در نظر گرفته شود.

۷.۱ معیارهای ارزیابی مختلف

در کد زیر، پیاده‌سازی معیارهای Net Profit، Sharpe Ratio، Sortino Ratio و Max Drawdown نشان داده شده است.

```

1 def NetProfit(series, init_fund=1000):
2     return series.iloc[-1] - init_fund
3
4
5 def SharpeRatio(series, risk_free_ratio=24):
6     mean = series.pct_change().mean() * 365 - risk_free_ratio
7     sigma = series.pct_change().std() * np.sqrt(365)
8     return mean / sigma
9
10
11 def SortinoRatio(series, risk_free_ratio=24):
12     stock = series.pct_change()
13     mean = stock.mean() * 365 - risk_free_ratio
14     std_neg = stock[stock < 0].std() * np.sqrt(365)
15     return mean / std_neg
16
17
18 def MaxDrawDown(series):
19     biggest_max = 1
20     lowest_min = 1
21     for day1 in range(len(series.index) - 1):
22         for day2 in range(day1 + 1, len(series.index)):
23             if series.iloc[day1] - series.iloc[day2] > biggest_max - lowest_min:
24                 biggest_max = series.iloc[day1]
25                 lowest_min = series.iloc[day2]
26     return (lowest_min - biggest_max) / biggest_max * 100

```

✓ 0.0s

شکل ۸: کد معیارهای Net Profit، Sharpe Ratio، Sortino Ratio، Max Drawdown

در کد زیر، حاصل این معیارها برای رمزارزها نشان داده شده است.

```

1 result = f"""BTC 1 day interval results are listed below
2     \tNet Profit : {NetProfit(BTC_1D_Result):.2f}
3     \tSharpe Ratio : {SharpeRatio(BTC_1D_Result):.2f}
4     \tSortino Ratio : {SortinoRatio(BTC_1D_Result):.2f}
5     \tMax Drawdown : {MaxDrawDown(BTC_1D_Result):.2f}
6 BTC 5 day interval results are listed below
7     \tNet Profit : {NetProfit(BTC_5D_Result):.2f}
8     \tSharpe Ratio : {SharpeRatio(BTC_5D_Result):.2f}
9     \tSortino Ratio : {SortinoRatio(BTC_5D_Result):.2f}
10    \tMax Drawdown : {MaxDrawDown(BTC_5D_Result):.2f}
11 ETH 1 day interval results are listed below
12    \tNet Profit : {NetProfit(ETH_1D_Result):.2f}
13    \tSharpe Ratio : {SharpeRatio(ETH_1D_Result):.2f}
14    \tSortino Ratio : {SortinoRatio(ETH_1D_Result):.2f}
15    \tMax Drawdown : {MaxDrawDown(ETH_1D_Result):.2f}
16 ETH 5 day interval results are listed below
17    \tNet Profit : {NetProfit(ETH_5D_Result):.2f}
18    \tSharpe Ratio : {SharpeRatio(ETH_5D_Result):.2f}
19    \tSortino Ratio : {SortinoRatio(ETH_5D_Result):.2f}
20    \tMax Drawdown : {MaxDrawDown(ETH_5D_Result):.2f}
21    """
22 print(result)

```

✓ 4.8s

شکل ۹: کد نشان دادن نتایج

```

BTC 1 day interval results are listed below
Net Profit : 1210.28
Sharpe Ratio : -56.60
Sortino Ratio : -89.29
Max Drawdown : -15.89
BTC 5 day interval results are listed below
Net Profit : 219.51
Sharpe Ratio : -56.61
Sortino Ratio : -50.18
Max Drawdown : -12.12
ETH 1 day interval results are listed below
Net Profit : 1282.75
Sharpe Ratio : -40.21
Sortino Ratio : -79.82
Max Drawdown : -23.27
ETH 5 day interval results are listed below
Net Profit : 103.45
Sharpe Ratio : -46.85
Sortino Ratio : -44.99
Max Drawdown : -10.30

```

شکل ۱۰: نتایج معیارها

۸.۱ تغییر در استراتژی

در این بخش، برخی موارد را تغییر دادیم تا استراتژی خود را بهبود بخشیم.

۱.۸.۱ تغییر در مقدار Oversold و Overbought

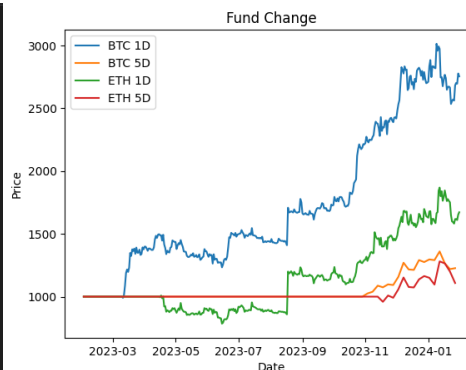
ابتدا مقادیر oversold و overbought را به ترتیب به 75 و 25 تغییر دادیم و نمودارها بصورت زیر تغییر کردند.

```

BTC 1 day interval results are listed below
Net Profit : 1753.44
Sharpe Ratio : -49.85
Sortino Ratio : -87.45
Max Drawdown : -15.90
BTC 5 day interval results are listed below
Net Profit : 226.65
Sharpe Ratio : -57.01
Sortino Ratio : -49.84
Max Drawdown : -10.36
ETH 1 day interval results are listed below
Net Profit : 671.45
Sharpe Ratio : -41.52
Sortino Ratio : -81.07
Max Drawdown : -15.34
ETH 5 day interval results are listed below
Net Profit : 108.52
Sharpe Ratio : -40.48
Sortino Ratio : -46.78
Max Drawdown : -13.43

```

(ب) نتایج متریک‌های مختلف



(آ) تغییر میزان سرمایه

۲ بخش دوم

۱.۲ تابع Backtest

در این تابع، ما استراتژی خواسته شده در بخش دوم را پیاده‌سازی می‌کنیم. لازم به ذکر است که اندیکاتور Bollinger Band در قسمت قبلی پیاده‌سازی شده است و از همان پیاده‌سازی استفاده می‌کنیم.

۱.۱.۲ بخش ابتدایی تابع

```
1 def backtest(stock, init_fund=1000, fund_rate=1):
2     bollinger_analysis = BollingerBand(stock.Close)
3
4     current_fund = init_fund
5     current_share = 0
6     sell_holding = False
7     sell_signal = False
8     buy_holding = False
9     buy_signal = False
10    sell_price = None
11
12    events_list = []
```

شکل ۱۱: بخش ابتدایی تابع Backtest

این تابع به عنوان ورودی، نمودار قیمت رمزارز، سرمایه اولیه و درصد ورود سرمایه را دریافت می‌کند که این موارد مشابه موارد متناظر در تابع backtest قبلی است. همچنین تعدادی متغیر تعریف شده است که در زیر تشریح می‌شوند.

- **bollinger_analysis** تحلیل اندیکاتور Bollinger Band بر روی نمودار قیمت رمزارز

- **current_fund** سرمایه فعلی

- **current_share** تعداد سهم‌های خریداری شده

- **sell_holding** آیا پوزیشن sell باز وجود دارد یا خیر

- **sell_signal** سیگنال فروش

- **buy_holding** آیا پوزیشن buy باز وجود دارد یا خیر؟

- **buy_signal** سیگنال خرید

- **sell_price** قیمت سهم هنگام بازکردن پوزیشن فروش

- **events_list** لیست زمان‌های بازکردن پوزیشن‌های خرید و فروش

۲.۱.۲ باز کردن پوزیشن Sell

```
18     if sell_signal and not sell_holding:
19         sell_signal = False
20         if buy_holding:
21             buy_holding = False
22             current_fund += current_share * stock.loc[day].Open
23             current_share = 0
24             sell_holding = True
25             current_share += current_fund / stock.loc[day].Open
26             current_fund -= current_share * stock.loc[day].Open
27             sell_price = stock.loc[day].Open
28             events_list.append(('s', day))
```

شکل ۱۲: بخش باز کردن پوزیشن فروش

در این بخش در شرط، ابتدا وجود سیگنال پوزیشن فروش و همچنین عدم وجود پوزیشن باز فروش را بررسی می‌کنیم. پس از ورود به بدنه شرط، سیگنال فروش (sell_signal) را برابر با False قرار می‌دهیم. به این معنی که سیگنال فروش دریافت شده است. سپس در بخش شرط داخلی، بررسی می‌کنیم آیا پوزیشن buy باز وجود دارد یا خیر. در صورت وجود، سهام‌ها را به قیمت روز به فروش می‌رسانیم. سپس متغیر sell_holding را برابر با True قرار می‌دهیم تا بگوییم پوزیشن باز فروش وجود دارد. در ادامه، تعداد سهم‌های ممکن را با توجه به سرمایه نقدی موجود، درصد ورود سرمایه و قیمت هر سهم محاسبه می‌کنیم. سپس قیمت سهم‌ها را از سرمایه نقدی موجود کم می‌کنیم. همچنین قیمت هر سهم را نیز ذخیره می‌کنیم تا در هنگام بستن پوزیشن فروش، بتوانیم سود و زیان را نیز محاسبه کنیم. در آخر نیز تاریخ بازکردن پوزیشن به همراه نوع پوزیشن را ذخیره می‌کنیم.

۳.۱.۲ باز کردن پوزیشن Buy

```

30     if buy_signal and not buy_holding:
31         buy_signal = False
32         if sell_holding:
33             sell_holding = False
34             current_fund += current_share * sell_price
35             current_fund += (sell_price - stock.loc[day].Open) * current_share
36             current_share = 0
37         buy_holding = True
38         current_share += current_fund / stock.loc[day].Open
39         current_fund -= current_share * stock.loc[day].Open
40         events_list.append(('b', day))

```

شکل ۱۳: بخش باز کردن پوزیشن خرید

این بخش نیز مشابه بخش قبلی می‌باشد.

۴.۱.۲ بررسی سیگنال‌های خرید یا فروش

در این بخش، سیگنال‌های خرید و فروش با توجه به اندیکاتور Bollinger Band تعیین می‌شوند. همچنین این بخش در آخر حلقه آمده است تا این سیگنال‌ها برای روز بعد اعمال شوند.

```

42     if stock.loc[day].Close > b_upper_bound:
43         sell_signal = True
44     else:
45         sell_signal = False
46     if stock.loc[day].Close < b_lower_bound:
47         buy_signal = True
48     else:
49         buy_signal = False
50
51     fund_per_day.loc[day] = current_fund + current_share * stock.loc[day].Close
52
53     return fund_per_day, events_list

```

شکل ۱۴: بررسی سیگنال خرید و فروش

در آخر نیز سرمایه هرروز محاسبه می‌شود. همچنین کل سرمایه در هرروز و همچنین تاریخ خرید و فروش‌ها نیز برگردانده می‌شوند.

۵.۱.۲ اجرای تابع Backtest

در این بخش، تابع backtest اجرا شده و نتایج نشان داده می‌شوند.

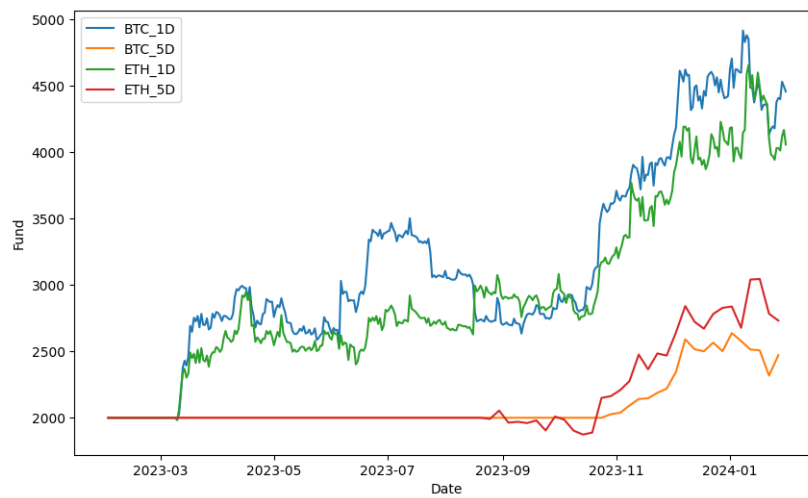
```
1 fund_rate = 1
2 init_fund = 2000
✓ 0.0s

1 BTC_1D_Result, BTC_1d_Events = backtest(
2     BTC_1d, init_fund=init_fund, fund_rate=fund_rate
3 )
4 BTC_5D_Result, BTC_5d_Events = backtest(
5     BTC_5d, init_fund=init_fund, fund_rate=fund_rate
6 )
7 ETH_1D_Result, ETH_1d_Events = backtest(
8     ETH_1d, init_fund=init_fund, fund_rate=fund_rate
9 )
10 ETH_5D_Result, ETH_5d_Events = backtest(
11     ETH_5d, init_fund=init_fund, fund_rate=fund_rate
12 )
✓ 2.6s
```

شکل ۱۵: اجرای تابع backtest

```
1 # Plotting Strategy Backtest
2 plt.figure(figsize=(10, 6))
3 plt.plot(BTC_1D_Result, label="BTC_1D")
4 plt.plot(BTC_5D_Result, label="BTC_5D")
5 plt.plot(ETH_1D_Result, label="ETH_1D")
6 plt.plot(ETH_5D_Result, label="ETH_5D")
7 plt.xlabel("Date")
8 plt.ylabel("Fund")
9 plt.legend()
10 plt.show()
✓ 0.5s
```

شکل ۱۶: کد ترسیم نتایج



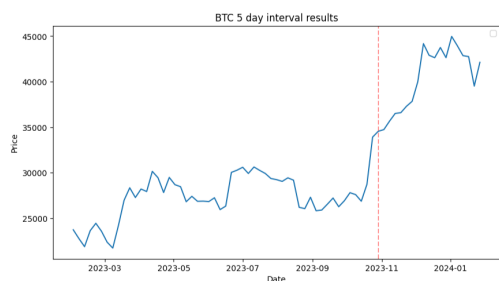
شکل ۱۷: نتایج ترسیم شده

```

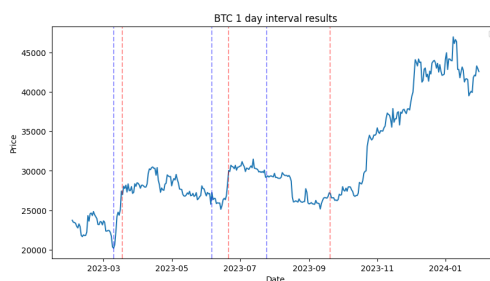
BTC 1 day interval results are listed below
  Net Profit : 2457.39
  Sharpe Ratio : -50.26
  Sortino Ratio : -82.36
  Max Drawdown : -24.81
BTC 5 day interval results are listed below
  Net Profit : 471.15
  Sharpe Ratio : -56.36
  Sortino Ratio : -50.03
  Max Drawdown : -12.12
ETH 1 day interval results are listed below
  Net Profit : 2057.93
  Sharpe Ratio : -50.08
  Sortino Ratio : -81.96
  Max Drawdown : -15.34
ETH 5 day interval results are listed below
  Net Profit : 731.95
  Sharpe Ratio : -33.60
  Sortino Ratio : -49.40
  Max Drawdown : -10.30

```

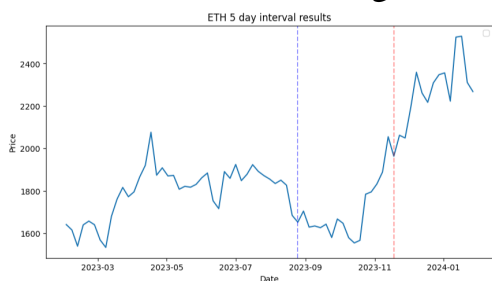
شکل ۱۸: نتایج متریک های مختلف



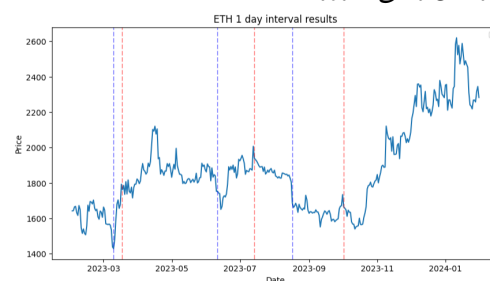
(ب) نمودار تاریخ خرید و فروش برای رمزارز BTC در بازه‌های زمانی ۵ روزه



(آ) نمودار تاریخ خرید و فروش برای رمزارز BTC در بازه‌های زمانی ۱ روزه



(د) نمودار تاریخ خرید و فروش برای رمزارز ETH در بازه‌های زمانی ۵ روزه



(ج) نمودار تاریخ خرید و فروش برای رمزارز ETH در بازه‌های زمانی ۱ روزه